

Problem Statement

Vision

Enable schools and social service teams to respond faster and more consistently to student concerns by centralising referrals, triage, case planning and follow-ups in a secure, auditable system with role-based access and clear ownership.

Problem statement

Student support and safeguarding workflows are time-sensitive and high-stakes, yet referrals, triage decisions, follow-ups and sensitive notes are often fragmented across forms, emails, spreadsheets and chats. This fragmentation creates delays, inconsistent triage decisions, unclear ownership of next actions, and increases the risk of over-sharing or under-sharing confidential information because access boundaries are not systematically enforced. Staff spend time searching for the latest status rather than intervening quickly. CaseHub provides a single, role-secured workflow from referral intake to intervention tracking with clear accountability, deadlines and an auditable history of key actions.

Target Users & Jobs-to-be-Done

- **Teacher/Form Teacher:** Submit a referral quickly; track status without seeing sensitive notes.
- **Counsellor/Social Worker:** Triage referrals, create cases, plan interventions, track tasks and record notes securely.
- **Safeguarding Lead:** Oversee high-risk cases and overdue follow-ups; monitor caseload and governance.
- **Programme Admin:** Maintain categories/templates/users/roles (minimal for MVP)

UI mock-up can consider using Figma

- Login
- Referral form
- Case list (filters by risk/overdue)
- Case details (tabs: plan / tasks / notes)
- Add session note
- Dashboard (caseload + due follow-ups)

Optional working software (20%) - If we choose to build:

- Frontend: React (fast)
- Backend: Node/NestJS (or Express)
- DB: Postgres

Deploy optional: Docker Compose “runs locally” demo

Roles

Name	Role	Responsibilities
Christine	Product Owner	Defines product vision and MVP scope; prioritises backlog (MoSCoW); writes/approves acceptance criteria; accepts/rejects completed stories during Sprint Review.
Sean	Scrum Master	Facilitates sprint ceremonies; removes blockers; protects sprint goal; ensures Jira hygiene (DoR/DoD usage, ticket status accuracy, sprint board cleanliness).
Kenny	Developer 1 – Full Stack (Frontend Owner & Integration Captain)	Owns UI implementation (screens, routing, component standards, accessibility); integrates frontend with APIs (auth, forms, role-based navigation); leads FE↔BE “glue fixes” during integration.
Ghim	Developer 2 – Backend Owner	Owns backend APIs and business rules; database schema/migrations; implements RBAC server-side, audit logging endpoints, and data integrity controls; builds dashboard aggregation APIs (counts/caseload/overdue).
Frances	Release & Documentation Lead / UAT Coordinator	Owns README + runbook + demo script; maintains sprint evidence packs (screenshots, links, notes); defines demo personas + seed-data checklist (teacher/counsellor/lead/admin scenarios); coordinates UAT walkthroughs and logs feedback/defects into Jira.
Sol	QA / Tester (QA Lead)	Defines test strategy (unit/integration/E2E); creates and runs test cases; regression testing; defect triage; validates demo/seed data and verifies acceptance criteria before PO sign-off.
Xiaorui	DevSecOps Engineer	Owns CI/CD pipeline and quality gates (lint/tests/secret & dependency scans); threat model + security test cases; Docker Compose technical setup (services, env vars, start/stop steps); release workflow for local demo.
Michelle	UI/UX Designer	Produces Figma mock-ups + user journeys; validates usability and consistency; supports Kenny with UI handoff specs (components, states, error messages).

Activity / Deliverable	PO	SM	Dev1 Full-stack/FE	Dev2 Back end	Release/Docs+UAT	QA	DevSec Ops	UX
1. Product vision, problem statement, MVP scope	A/R	C	C	C	I	I	C	C
2. Backlog creation (Epics/Stories) + MoSCoW priority *(amended: PO owns)	A/R	C	C	C	C	C	C	C
3. Acceptance criteria definition & story readiness (DoR) *(amended: SM facilitates;	A	C	R	R	C	R	C	C
4. Sprint planning (goal, capacity, commitment, sprint backlog)(<i>amended: Devs own commitment; SM facilitates</i>)	A	R	R	R	C	C	C	I
5. Daily stand-ups & impediment removal	I	A/R	R	R	I	R	R	I
6. Backlog refinement session (mid-sprint) *(amended: Devs share	A	R	R	R	C	C	C	C

7. UX flows & Figma mock-ups(<i>amended: UX owns design deliverable; PO provides product input</i>)	C	I	C	I	C	C	I	A/R
8. UI implementation (screens, routing, role-based navigation)	C	I	A/R	C	I	C	C	C
9. Backend implementation (APIs, business rules, schema/migrations)	C	I	C	A/R	I	C	C	I
10. RBAC server-side enforcement (permissions/guards)(<i>amended: backend owns; DevSecOps consulted</i>)	C	I	C	A/R	I	C	C	I
11. Audit logging + data integrity controls <i>*(amended</i>	C	I	C	A/R	I	C	C	I
12. CI/CD pipeline + security gates (secret scan, dependency scan, tests)	I	I	C	C	I	C	A/R	I
13. Docker Compose technical setup (services, env vars, start/stop)	I	I	C	C	C	I	A/R	I

14. README + runbook + demo script + evidence pack *(minor	C	I	I	I	A/R	C	C	C
15. Test strategy (unit/integration/E2E) + test cases	C	I	C	C	I	A/R	C	C
16. Regression testing + defect triage workflow	I	C	R	R	C	A/R	C	I
17. Sprint review (demo) + PO acceptance	A	R	R	R	C	C	C	I
18. Sprint retrospective + improvement actions tracking(<i>tightened: SM owns process; others contribute</i>)	C	A/R	R	R	R	R	R	I
19. Final demo recording + submission package *(amended: Release owns packaging	C	C	R	R	A/R	C	C	C
20. Estimation & sizing (story points/effort)	C	R(<i>facilitates</i>)	R	A/R	I	C	C	I
21. NFR outcomes + verification approach (performance/availability/observability/security)	C	C	R	R	C	R	A/R	C

22. Data handling rule for artefacts & non-prod (synthetic data only, redaction for evidence packs)	A	C	C	C	R	C	R	I
23. Threat modelling + security review (RBAC, audit log, endpoints)	C	I	C	R	I	C	A/R	I
24. Release readiness checklist + go/no-go decision	C	C	C	C	A/R	C	C	I
25. Change control / decision log discipline (scope changes recorded, rationale captured)	A/R	C	I	I	C	I	C	I

Legend:

- **R = Responsible** (does the work)
- **A = Accountable** (final owner / sign-off)
- **C = Consulted** (provides input)
- **I = Informed** (kept updated)

Sprints Deliverables

Sprint 1 — Referral Intake (FOUNDATION FLOW)

Goal:

Teacher can submit a referral and counsellor can see it in a queue.

Deliverables

1. MVP flow definition (first slice)

- Teacher → Submit referral → Status visible
- Counsellor → View referral queue

2. Backlog definition

- Top **8–10 backlog items only (EP-REF + EP-TRI partial)**
- Must include:
 - CH-001 Submit referral
 - CH-002 View status
 - CH-003 Referral queue
 - CH-004 (draft only, no full logic yet)

3. Acceptance Criteria (AC)

- Referral creation works end-to-end
- Teacher sees only own referrals
- Counsellor sees all referrals in queue
- No triage logic yet enforced (just placeholder state)

4. Demo Script v1

“Teacher submits referral → appears in counsellor queue”

Sprint 1 Exit Condition

- A referral exists in system
 - It is visible to correct roles
 - No case or triage logic required yet
-

Sprint 2 — Triage + Case Creation (CORE DECISION FLOW)

Goal:

Counsellor can triage referrals and convert them into cases.

Deliverables

1. Triage definitions

- Risk levels:
 - Low / Medium / High
- Outcomes:
 - Create Case / Monitor / Close

2. Case definitions

- Case statuses:
 - Open / In Progress / Closed
- Case linked to referral

3. Backlog scope

- CH-004 (FULL IMPLEMENTATION)
- CH-005 Convert referral → case
- CH-006 Case status lifecycle

4. Acceptance Criteria

- Triage saves risk + outcome
- Referral status updates after triage
- “Create Case” generates case linked to referral
- Case visible in case list
- Status updates persist

5. Demo Script v2

“Counsellor triages referral → creates case → case appears in case list”

Sprint 2 Exit Condition

- Full referral → triage → case lifecycle works

Sprint 3 — Tasks & Intervention Planning

Goal:

Cases become actionable via tasks and deadlines.

Deliverables

1. Task model definitions

- Status:
 - Open / Done
- Rules:
 - Due date required
 - Assigned user required

2. Overdue rules

- Overdue = due date < today AND not Done

3. Backlog scope

- CH-007 Task creation
- CH-008 Overdue indicators

4. Acceptance Criteria

- Tasks can be created per case
- Tasks require due date + assignee
- Tasks appear in case view
- Overdue tasks are flagged
- Overdue count visible (basic UI only, dashboard later refined)

5. Demo Script v3

“Case → create tasks → see overdue indicator”

Sprint 3 Exit Condition

- Case has actionable task layer
 - Overdue logic works reliably
-

Sprint 4 — Notes + Dashboard (VISIBILITY + OVERSIGHT)

Goal:

Secure case documentation + leadership visibility.

Deliverables

1. Notes rules

- Visible to:
 - Assigned counsellor
 - Lead
- NOT visible to teacher

2. Dashboard definitions

- Risk summary:
 - High / Med / Low case counts
- Overdue tasks count

3. Backlog scope

- CH-009 Notes system
- CH-010 Lead dashboard

4. Acceptance Criteria

- Notes can be created per case
- Notes stored chronologically
- Teacher cannot access notes
- Lead dashboard shows:
 - risk distribution
 - overdue tasks count

5. Demo Script v4

“Case → add notes → Lead views dashboard insights”

Sprint 4 Exit Condition

- Full visibility layer works (secure + summarised)

Sprint 5 — Hardening + Final MVP Delivery

Goal:

Stabilise system, enforce security, prepare final demo + submission.

Deliverables

1. Final MVP scope lock

- Freeze features
- Remove incomplete or unused flows

2. Security hardening

- Full RBAC enforcement
- Teacher restrictions verified
- Case/notes isolation confirmed

3. Audit logging (FINALISE CH-012)

- Logs:
 - triage updates
 - case creation
 - task creation
 - note creation

4. UAT scenarios

- End-to-end test pack:
 - referral → case → task → note → dashboard

5. Final demo script

- Complete flow:
 - Teacher submits referral
 - Counsellor triages
 - Case created
 - Task added
 - Note added
 - Lead reviews dashboard

6. Final report sections

- Vision
- Problem statement alignment
- Scope (MVP boundaries)
- Backlog rationale (why each feature exists)
- Acceptance criteria summary
- Demo walkthrough narrative

Front + Backend Developer

Backend Development

Scope	Date to Complete	Notes
Database schema design		
REST API endpoint design		
RBAC structure		
Authentication (login + JWT)		
Build API endpoints		
Connect to Postgres		
Unit tests for APIs		
Document backend architecture + API design for report		

Frontend Development

Scope	Date to Complete	Notes
Database schema design		
REST API endpoint design		
RBAC structure		
Authentication (login + JWT)		
Build API endpoints		
Connect to Postgres		
Unit tests for APIs		
Document backend architecture + API design for report		

Tool/Software	What It's For	Where to Get
VS Code 🐼	Code editor	code.visualstudio.com
Node.js (LTS)	We need to use Node js as a module. Try to understand this before the meeting	nodejs.org
npm	Package manager (comes with Node)	Included with Node.js

Express	Backend framework	<code>npm install express</code>
Supabase	Database	https://supabase.com/
pgAdmin	GUI to manage Postgres DB	pgadmin.org
Postman	Test your API endpoints To check the status	postman.com
Git	Version control	git-scm.com
Github	Remote repo + collaboration	github.com

What i need to learn	Where to learn	What is it for
React Basics	https://javascript.info/	
Node.js + Express	https://www.youtube.com/watch?v=fBNz5xF-Kx4	setting up a server, routing, middleware
REST APIs	https://www.youtube.com/watch?v=SLwpgD8n3d0	GET, POST, PUT, DELETE endpoints
Supabase	tables, relationships, basic queries	Database, tables, relationships, basic queries
JWT Authentication	"JWT authentication Node Express tutorial" on YouTube	login flow, token generation, protected routes
React language		

Sprint 1	Understand User's Stories: - Michelle to come up with the user story - We will have several version: version control - Need Xiao Rui to tell us what security consideration we need to implement -
-----------------	---

Notes

- Docker: create code space for us
- Everything will run on github codespaces
- Database runs locally
- Kenny to create the logic flow
- Ask Kelvin if we can use Supabase, BaaS

MVP Flow / Deliverable

MVP core workflow

1. Teacher logs in → submits referral
2. Counsellor views referral queue → triages risk (Low/Med/High)
3. Counsellor converts referral → case and assigns owner
4. Counsellor creates follow-up tasks with due dates → overdue indicator works
5. Counsellor logs session notes (restricted access)
6. Lead dashboard shows risk counts + overdue follow-ups + caseload by owner
7. Security proof: teacher blocked from notes; audit log captures triage/case/note actions

Status Rules (MVP)

- 1) Referral status is system-managed and follows: Submitted → In Triage → Case Opened → Closed.
- 2) “Submitted” is set when a teacher successfully submits a referral.
- 3) “In Triage” is set when a counsellor records triage (risk/outcome) for the referral.
- 4) “Case Opened” is set when a counsellor converts a referral into a case.
- 5) “Closed” is set when the associated case is closed (high-level status for teacher tracking).
- 6) Case status is owner-managed (Counsellor/Lead as permitted) and follows: Open → In Progress → Closed.
- 7) Task status supports: Open / Done; overdue is derived by the system when Today > DueDate AND status ≠ Done.
- 8) Teachers see referral status timeline only; they must not see triage details, notes, dashboard or audit logs.

Success Criteria (for demo)

- End-to-end flow demonstrable with seeded accounts and data
- MVP Flow / Deliverable and User JourneyRole-based access enforced (teacher cannot access notes)
- Audit log shows key actions with actor + timestamp
- Dashboard shows accurate counts aligned to seeded data
- Jira backlog and sprint boards show transparent Agile execution

Deliverable: MVP User Journeys (Web + Mobile Mock)

CaseHub — MVP User Journeys (v1.0)

Web Journey A — Teacher Referral (Status-only)

Goal: Submit a concern and track status without seeing sensitive content.

Steps:

1. Login
2. “Create Referral” → enter student identifier, category, urgency, description → submit
3. “My Referrals” → view list with status (Submitted / In Triage / Case Opened / Closed)
4. Open a referral → see status timeline only (no triage notes, no counselling notes)

Web Journey B — Counsellor Triage → Case → Tasks → Notes

Goal: Turn referrals into coordinated interventions.

Steps:

1. Login
2. "Referral Queue" → filter by urgency/date/status
3. Open referral → set risk level (Low/Med/High) + triage outcome → save
4. "Create Case" → assign owner → set case status
5. Add tasks (title, due date, assignee) → mark complete → observe overdue badges
6. Add session note (restricted) → view note chronology
7. Case page shows: status, tasks, notes, audit events (lead/admin only for audit view)

Web Journey C — Safeguarding Lead Oversight

Goal: Monitor risk and overdue follow-ups.

Steps:

1. Login
2. "Dashboard" → counts by risk + overdue follow-ups + caseload per owner
3. Drill into list of high-risk or overdue cases
4. Review audit log (optional view) for governance

Mobile Mock (Figma only) — Key Flows

Teacher mobile mock: Quick referral creation + "My referrals" status list

Counsellor mobile mock: Quick queue view + triage + case "at-a-glance" (tasks due, last note timestamp)

Lead mobile mock: Dashboard snapshot (risk counts + overdue)

Backlog Items

Deliverable: Top 12 Backlog Items to kick off project

EP-REF — Referral Intake

CH-001 (Must) Teacher submits referral

As a Teacher, I want to submit a referral with required details so that the student support team can respond quickly.

Acceptance Criteria:

- Given teacher is logged in, when required fields are provided and submitted, then referral is created with ID and timestamp.
- Validation errors shown for missing required fields.

CH-002 (Must) Teacher views status-only referral list

As a Teacher, I want to view the status of my submitted referrals so that I know support is in progress without seeing sensitive information.

Acceptance Criteria:

- Teacher sees own referrals they submitted.
- Each referral shows status (e.g. Submitted / In Review / Closed)
- Teacher cannot view triage details or notes.

EP-TRI — Triage & Risk

CH-003 (Must) Counsellor referral queue

As a Counsellor, I want to view all referrals in a queue so that I can triage efficiently.

Acceptance Criteria:

- Queue displays list of referrals with date and status.
- Filtering by status returns only matching referrals

CH-004 (Must) Triage risk level + outcome

As a Counsellor, I want to assign a risk level and triage outcome to a referral so that it can be prioritised and acted on quickly.

Acceptance Criteria:

Risk level assignment

- Given a referral is opened, when the counsellor completes triage, then a risk level (Low / Medium / High) must be selected before saving.

Triage outcome selection

- Given a referral is triaged, when saved, then a triage outcome must be selected (Create Case / Monitor / Close).

Mandatory note for higher risk

- If “High” risk is selected, a short justification note is required before submission.

Edit capability

- Counsellor can update risk level and outcome after initial submission.

Access control

- Triage details are visible only to Counsellor and Lead roles.
- Teacher cannot view triage details.

Data persistence

- Risk level and outcome are saved and displayed in the referral view.
- Saving triage updates the referral status:
 - When triage is first saved, status becomes “In Review”.
 - If outcome is “Close”, status becomes “Closed”.

EP-CASE — Case Management

CH-005 (Must) Convert referral to case + assign owner

As a Counsellor, I want to create a case from a referral and assign an owner so that accountability is clear.

Acceptance Criteria:

- Counsellor can create a case from a referral.
- Case must have an assigned owner before creation.
- Case is linked to the original referral..

CH-006 (Must) Case status lifecycle

As a Counsellor, I want to update case status (Open/In Progress/Closed) so that progress is trackable.

Acceptance Criteria:

- Case status supports Open / In Progress / Closed.
- Status updates are saved and reflected in the case view..

EP-PLAN — Tasks

CH-007 (Must) Create tasks with due dates

As a Counsellor, I want to create follow-up tasks with due dates and assign them to a user so that actions are clearly owned and tracked.

Acceptance Criteria:

Task creation

- Given a case is open, when a counsellor creates a task, then task must include:
 - Title (required)
 - Due date (required)
 - Assigned user (required)

Task status

- Task status supports Open/Done.
- Default status is Open upon creation.

Task visibility

- Tasks are visible to the assigned user, case owner, and Lead.
- Tasks appear within the case detail view.

Edit capability

- Counsellor can edit task details (title, due date, status).

Data persistence

- Task details and status updates are saved and reflected consistently in the case view.

CH-008 (Must) Overdue indicators

As a Counsellor/Lead, I want overdue tasks clearly flagged so that I can prioritise follow-ups.

Acceptance Criteria:

Overdue logic

- A task is marked as overdue when:
 - Due date has passed AND
 - Task status is not "Done"

Visual indicator

- Overdue tasks display a clear overdue indicator in the case view.

Dashboard visibility

- Overdue task count is displayed on the Lead dashboard.

EP-NOTES — Session Notes

CH-009 (Must) Counsellor notes (restricted)

As a Counsellor, I want to add secure notes to a case so that case history is recorded and accessible only to authorised users.

Acceptance Criteria:

Note creation

- Given a counsellor is viewing an assigned case, when they add a note, then the note must include:
 - Note content (required)
 - Automatically captured author and timestamp

Note visibility

- Notes are visible only to:
 - Assigned counsellor
 - Case owner
 - Lead role
- Teacher role cannot view notes (access denied).

Case-level access

- Counsellor can only create and view notes for cases they are assigned to.

Data persistence

- Notes are saved and displayed in chronological order within the case view.

Audit logging

- Creating a note generates an audit log with user ID and timestamp.

EP-DASH — Dashboard

CH-010 (Must) Lead dashboard

As a Safeguarding Lead, I want a dashboard showing risk and overdue follow-ups so that I can oversee urgency and workload.

Acceptance Criteria:

- Dashboard displays:
 - Number of cases by risk level
 - Number of overdue tasks .
- Teacher cannot access dashboard. Only Lead role can access dashboard

EP-SEC — Security & DevSecOps

CH-011 (Must) RBAC enforcement - Unauthorised users must not access protected resources.

As a System, I want role-based access enforced (Unauthorised users must not access protected resources) so that sensitive data is protected.

Acceptance Criteria:

- Teacher cannot access triage details or notes.
- Counsellor can only access cases they are assigned to.
- Only Lead role can access the dashboard.

CH-012 (Must) Audit logging for sensitive actions

As a Lead/Admin, I want to view an audit log of key actions so that changes to referrals, cases, and notes are traceable.

Acceptance Criteria

Audit event creation

- The system logs an audit event for the following actions:
 - Triage created or updated
 - Case created
 - Note created

Audit event details

- Each audit event captures:
 - Action type (e.g., "Triage Updated", "Case Created")
 - Associated record ID (referral or case)
 - User who performed the action
 - Timestamp of the action

Audit log access

- Audit logs are viewable only by Lead and Admin roles.
- Counsellor and Teacher roles cannot access audit logs.

Audit log view

- Audit events are displayed in chronological order.

Data persistence

- Audit events are stored and cannot be edited or deleted by users.

Jira backlog structure

EPIC 1 — Referral Intake (EP-REF)

CH-001 — Teacher submits referral

Story:

As a Teacher, I want to submit a referral so that student support can be triggered quickly.

Sub-tasks:

- Build referral creation form (frontend)
- Create referral API endpoint (backend)
- Validate required fields
- Store referral with ID + timestamp
- Return success response
- Show validation errors on UI

CH-002 — Teacher views referral status

Story:

As a Teacher, I want to view only the status of my submitted referrals.

Sub-tasks:

- Build “My Referrals” UI list
- API to fetch teacher’s referrals only
- Display status field only (no sensitive data)
- Implement teacher-level access control

EPIC 2 — Triage & Risk (EP-TRI)

CH-003 — Counsellor referral queue

Story:

As a Counsellor, I want to view a queue of referrals to triage them efficiently.

Sub-tasks:

- Build counsellor referral queue UI
- API to fetch all referrals
- Add status + date sorting
- Implement status filter

CH-004 — Triage risk level + outcome

Story:

As a Counsellor, I want to triage referrals so they can be prioritised.

Sub-tasks:

- Build triage form UI
- Add risk level selection (Low/Med/High)
- Add outcome selection (Create Case / Monitor / Close)
- Add mandatory note for High risk
- Update referral status on save (Submitted → In Review / Closed)
- Save triage data to DB
- Restrict access to Counsellor + Lead

EPIC 3 — Case Management (EP-CASE)

CH-005 — Convert referral to case + assign owner

Story:

As a Counsellor, I want to convert a referral into a case with ownership.

Sub-tasks:

- Create “Convert to Case” action
- Create case record linked to referral
- Assign case owner
- API for case creation
- Display case in case list

CH-006 — Case status lifecycle

Story:

As a Counsellor, I want to track case progress.

Sub-tasks:

- Build case status UI (Open/In Progress/Closed)
- API to update case status
- Persist status changes
- Display status in case view

EPIC 4 — Tasks (EP-PLAN)

CH-007 — Create tasks with due dates

Story:

As a Counsellor, I want to create tasks so actions are tracked.

Sub-tasks:

- Build task creation UI
- Add title, due date, assigned user fields
- Save task to database
- Display tasks in case view
- Assign task ownership
- Task status default = Open

CH-008 — Overdue task indicators

Story:

As a Counsellor/Lead, I want overdue tasks highlighted.

Sub-tasks:

- Backend logic for overdue detection
- UI badge for overdue tasks
- Add overdue count to dashboard
- Filter overdue tasks

EPIC 5 — Notes (EP-NOTES)

CH-009 — Counsellor notes (restricted)

Story:

As a Counsellor, I want to add private case notes.

Sub-tasks:

- Build notes input UI
- Save note with author + timestamp
- Display notes in chronological order
- Restrict access (Counsellor/Lead only)
- Block Teacher access
- Store notes in DB

EPIC 6 — Dashboard (EP-DASH)**CH-010 — Lead dashboard****Story:**

As a Lead, I want visibility of risk and overdue workload.

Sub-tasks:

- Build dashboard UI
- API for risk aggregation
- API for overdue tasks count
- Display charts/cards (simple counts only)
- Restrict access to Lead only

EPIC 7 — Security (EP-SEC)**CH-011 — RBAC enforcement****Story:**

As a system, I want role-based access control.

Sub-tasks:

- Implement authentication middleware
- Define roles (Teacher, Counsellor, Lead)
- Block Teacher access to notes & triage
- Restrict case access to assigned counsellor
- Restrict dashboard to Lead

CH-012 — Audit logging**Story:**

As a Lead/Admin, I want traceability of key actions.

Sub-tasks:

- Create audit log table/schema
- Log triage updates
- Log case creation
- Log note creation
- Store user + timestamp + action + record ID
- Build simple audit log viewer
- Restrict access to Lead/Admin only
- Ensure logs are immutable

EPIC 8 — Dev Setup (NON-FEATURE, optional)

CH-013 — Seed demo data (optional dev support)

Sub-tasks:

- Create demo users (teacher/counsellor/lead)
- Seed sample referrals
- Seed sample cases/tasks

Core workflow (Sprint 1)

- Referral → Triage → Case

Support layer (Sprint 2)

- Tasks → Notes → Dashboard → Audit

Security layer (continuous)

- RBAC + Audit logs

Data Dictionary

Deliverable: Data Dictionary v1 (Minimum fields)

- what data exists, what each field means, and how it should be used

Referral (what was reported)

- referralId: string (UUID)
- studentName: string
- category: string (optional MVP classification)
- urgency: Low | Medium | High (optional input, can default Low)
- description: text
- createdBy: userId (Teacher)
- createdAt: datetime
- status: Submitted | In Review | Closed

Triage (risk and decision layer)

- triageId: string (UUID)
- referralId: string (UUID)
- riskLevel: Low | Medium | High
- triageOutcome: Create Case | Monitor | Close
- triageNotes: text (RESTRICTED)
- triagedBy: userId (Counsellor)
- triagedAt: datetime

Case (ownership & execution tracking)

- caseId: string (UUID)
- referralId: string (UUID)
- ownerId: userId (Counsellor)
- status: Open | In Progress | Closed
- riskLevel: Low | Medium | High (copied from triage)
- createdAt: datetime
- updatedAt: datetime

Task (follow-ups + execution control)

- taskId: string (UUID)
- caseId: string (UUID)
- title: string
- dueDate: date
- status: Open | Done
- assignedTo: userId
- createdAt: datetime
- updatedAt: datetime

Note (restricted clinical style documentation)

- noteId: string (UUID)
- caseId: string (UUID)
- noteText: text
- authorId: userId
- createdAt: datetime

- updatedAt: datetime

Audit Log (traceability and compliance)

- auditId: string (UUID)
- actionType:
 - - TRIAGE_CREATED
 - - TRIAGE_UPDATED
 - - CASE_CREATED
 - - TASK_CREATED
 - - TASK_UPDATED
 - - NOTE_CREATED
 - - NOTE_UPDATED
- actorId: userId
- targetType: Referral | Case | Task | Note
- targetId: string (UUID)
- timestamp: datetime

Metadata Rule

Allowed:

- field names changed
- status transitions (e.g. Open → In Progress)
- IDs
- counts
- system correlation IDs

NOT allowed:

- noteText
- triageNotes
- free-text descriptions
- counselling content
- before/after sensitive data

Security + Access

CaseHub — RBAC & Privacy Rules (v1.0)

Roles

- Teacher
- Counsellor
- Safeguarding Lead
- Admin (optional for MVP setup only)

1. What each role can DO (Actions)

Teacher

- Create referral
- View own referrals (status-only view)
- Cannot:**
- Triage referrals
- Create or view cases
- Create or view tasks
- Create or view notes
- Access dashboard
- Access audit logs
- Access any user data beyond own referrals

Counsellor

- View referral queue
- Perform triage (risk + outcome)
- Convert referral → case
- View assigned cases only
- Create / edit tasks (within assigned cases)
- Create / view notes (within assigned cases)
- Cannot:**
- View audit logs
- View unassigned cases
- View teacher-only restricted data beyond referral metadata

Safeguarding Lead

- View all cases
- View dashboard (risk + workload + overdue)
- View audit logs
- View all notes and triage decisions (read-only unless explicitly allowed)
- Optional (MVP-light):**
- Reassign case owner
- Override case priority (if needed for escalation workflows)

Admin (optional MVP setup role only)

- Manage users (create/disable users)
- Assign roles

- Manage categories/templates (if implemented)
- View audit logs

Must NOT:

- Access counselling notes content as part of workflow (unless also Lead)
- Modify case data unless explicitly dual-role

2. What each role can SEE (Data visibility)

Teacher sees:

- Own referrals
- Referral status timeline only:
 - Submitted
 - In Review
 - Closed

Teacher must NEVER see:

- Triage risk level
- Triage notes or rationale
- Case data
- Tasks
- Notes
- Dashboard
- Audit logs

Counsellor sees:

- Full referral details (assigned queue)
- Triage fields
- Cases they own
- Tasks in their cases
- Notes in their cases

Lead sees:

- All referrals and cases
- All triage data
- All tasks (read access)
- Dashboard analytics
- Audit logs

Admin sees:

- System configuration data only
- User/role mappings
- Audit logs

3. Data Sensitivity Classification (MVP)

Highly Sensitive

- Counselling notes
 - Triage rationale notes
 - Free-text case notes containing personal information
- Strict RBAC + audit required

Medium Sensitivity

- Risk level (Low/Med/High)
- Case status
- Assigned counsellor/owner
- Task assignments

Low Sensitivity

- Referral submission data (basic metadata)
- Status labels
- Timestamps
- System-generated IDs

4. Security Acceptance Rules

Teacher accessing `/notes` or `/cases` returns **Access Denied (no data leakage)**

Counsellor cannot access cases not assigned to them

Lead can access all cases and dashboard

Only Lead/Admin can access audit logs

All sensitive actions generate audit log entry:

- triage updates
- case creation
- task creation
- note creation

Data Lifecycle, Retention & Governance — MVP-level requirements

Retention (MVP placeholder rule)

- All entities (referrals, cases, tasks, notes, audit logs) are retained indefinitely in MVP database
- Retention policy to be defined later as a backlog item

Access Review

- Role changes must be logged in audit system
- Only Admin/Lead can modify roles
- All role changes must generate audit event:
 - actor
 - before/after role
 - timestamp

Deletion / Closure Rules

- Cases are **not physically deleted in MVP**
- Closed cases become **read-only state**
- Notes and audit logs are immutable

Sprint 1 and 2 Goals

Sprint 1— Core Flow (Referral → Triage → Case)

Sprint Goal

Get the **end-to-end flow working**: teacher submits → counsellor triages → case is created.

Build Items

EP-REF

- CH-001 Teacher submits referral
- CH-002 Teacher views status-only list

EP-TRI

- CH-003 Counsellor referral queue
- CH-004 Triage risk + outcome (basic version + status update)

EP-CASE

- CH-005 Convert referral to case + assign owner
- CH-006 Case status lifecycle

EP-SEC (partial)

- CH-011 RBAC (ONLY basic enforcement)

Sprint 1 Outcome

- Teacher submits referral
- Counsellor sees queue
- Counsellor triages referral
- Referral becomes case
- Case has owner + status

Sprint 2 — Accountability + Visibility Layer

Sprint Goal

Add **tracking, follow-ups, and oversight**

Build Items

EP-PLAN

- CH-007 Tasks with due dates + assigned user
- CH-008 Overdue indicators

EP-NOTES

- CH-009 Counsellor notes (restricted)

EP-DASH

- CH-010 Lead dashboard

EP-SEC

- CH-012 Audit logging (full)

Sprint 2 Outcome

- Assign tasks to users
- Track overdue actions
- Add secure case notes
- Lead sees risk + overdue overview
- System logs key actions

Scope Guardrails

Scope Guardrails | estimate based on 5-Week MVP

MUST HAVE (Non-negotiable MVP Demo Scope)

These are **required for a working end-to-end system demo**. If any are missing, MVP is incomplete.

1. Authentication + Role-Based Access (RBAC core)

- Login system (basic authentication)
- Roles:
 - Teacher
 - Counsellor
 - Safeguarding Lead
 - Admin-lite (configuration only, no workflow access expansion)

Must enforce:

- Role-based routing and API protection
- Default deny for unauthorised access

2. Referral Intake (Teacher)

- Create referral
- View “My Referrals” only
- Status-only visibility:
 - Submitted
 - In Review
 - Closed

3. Referral Queue (Counsellor)

- View all referrals in queue
- Open referral details
- Basic filtering by status (optional but included in MVP core usability)

4. Triage (Counsellor)

- Set risk level:
 - Low / Medium / High
- Set triage outcome:
 - Create Case / Monitor / Close
- Save triage decision

5. Case Creation (System-triggered flow)

- Convert referral → case (based on triage outcome)
- Assign case owner (Counsellor)
- Case linked to referral

6. Case Management (Core lifecycle)

- Case status:
 - Open / In Progress / Closed
- Status updates persist and are role-restricted

7. Tasks (Follow-ups)

- Create task per case
- Fields:
 - Title
 - Due date
 - Assigned user
- Status:
 - Open / Done
- Overdue indicator:
 - dueDate < today AND not Done

8. Notes (Restricted access)

- Create notes on case
- View rules:
 - Counsellor (assigned cases only)
 - Lead (all cases)
 - Teacher (NO ACCESS)

9. RBAC Enforcement (System-wide)

Must enforce:

- Teacher cannot access:
 - cases
 - notes
 - triage data
 - dashboard
- Counsellor restricted to assigned cases
- Lead has global visibility
- All endpoints protected (not just UI)

10. Audit Logging (Security-critical)

Must log:

- Triage updates
- Case creation
- Task creation/update
- Note creation/update

Each log must include:

- actorId
- actionType
- targetId
- timestamp

11. Lead Dashboard (MVP Insights)

Must show:

- Cases by risk level
 - Overdue tasks count
 - Caseload by counsellor
-

SHOULD HAVE (High value, improves usability but not required for MVP success)

1. Case filtering (Counsellor + Lead)

- Filter by:
 - risk level
 - status
 - overdue tasks

2. Basic notifications (in-app only)

- Overdue task alert banner
- No email/SMS required

3. CI/CD quality gates

- Lint checks
- Basic test execution
- Dependency/security scan (block merge on failure)

4. Seeded demo data

- teacher1 / counsellor1 / lead1 / admin1
 - sample referrals + cases + tasks
-

COULD HAVE (Only if ahead of schedule)

1. Note editing (optional enhancement)

- Edit notes allowed
- Must generate audit log entry

2. Dashboard drill-down

- Click metric → view underlying cases list

3. Referral attachments

- Upload optional evidence files

4. UI polish improvements

- Empty states
- Better loading states

- Component consistency improvements

5. Mobile UX mock improvements

- Additional Figma screens beyond core flows
 - No production mobile app required
-

WON'T HAVE (Explicitly out of scope for MVP)

1. Production-grade infrastructure

- No enterprise hosting guarantees
- No multi-region setup
- No SSO (Google/Microsoft login)

2. External integrations

- No MIS systems
- No external agency data exchange
- No government/school system sync

3. Advanced admin system

- No full admin console
- No workflow builder
- No template engine

4. Advanced analytics / AI

- No predictive risk scoring
- No machine learning models
- No behavioural analytics

5. Full mobile application

- Web-only system
- Mobile = mock/prototype only

6. Export / reporting packs

- No PDF case packs
- No legal export bundles
- No redaction workflows

Meeting Agenda

